

TO DO (28/08/25):

- Read Consistent zoomout
https://www.lix.polytechnique.fr/~maks/papers/ConsistentZoomOut_SGP2020.pdf
- Project Repo: https://github.com/ruqihuang/SGP2020_ConsistentZoomOut
- Look at how feature correspondence is formulated in the functional maps -> we want to use the Knn neighbours as a regularisation term in the optimisation

Consistent Zoomout Structure:

1. Load meshes
 - a. with their Laplacian Basis functions
 - b. subsample to make it faster (do i need to do that?)
2. Load initial maps
 - a. Actually I think they have pointwise maps but of sampled points and then they compute the dense pointwise maps at the end after optimisation
3. Create initial map network
4. Run ConsistentZoomout with: zoomout from dim 30 to 80, with step size 2

Important Info Needed for Evaluation

- Use their Evaluation metrics and baselines to test?

Metrics. In our tests, we mainly measure the accuracy w.r.t. the given ground-truth, the consistency error, and the runtime of the computed maps from different baselines.

- **Accuracy:** we measure the geodesic distance from the mapped vertex to the given ground-truth as the accuracy evaluation.
- **Bijectivity/Consistency:** with the computed maps T_{ij} from shape S_i to shape S_j , we take the compound map $T_{ji} \circ T_{ij}$ as a map from S_i to itself. We then measure the Euclidean distance between this compound map to the identity map as the consistency or bijectivity error.
- **Runtime:** we also evaluate the average runtime per shape pair for different baselines.

Initialization & Parameters. As our method is one of map synchronization, it requires a set of approximate initial maps as input. We provide the details on the exact procedure for initialization along with all of the parameters used in our implementation in Appendix D. We will release a complete implementation of our method upon acceptance.

Appendix D: Initialization & Parameters

Initialization.

In our tests, we used the same initialization for different baseline methods.

- **SHREC'07:** for the tests involving this dataset, we always use the maps computed using BIM [KLF11] as initialization.
- **SCAPE (dense):** we pick 18 shapes from this dataset, and use a learning-based framework, SURFMNet [RSO19], to compute the densely pairwise maps as initialization.
- **FAUST (sparse):** we use the provided maps in [RPWO18], which contains 300 pairs of maps among the whole 100 shapes.
- **SHREC'19:** we use the provided maps in [MMR⁺19] as initialization, which are computed from 5 given landmarks.

Parameters. Throughout all the experiments, we use the same set of parameters except for the way of augmenting dimension of functional maps: For computing the canonical consistent latent basis, we consistently use the first $[0.9k]$ canonical consistent latent basis, where k is the dimension of functional maps at current iteration. We use Euclidean farthest point sampling starting from a random seed point, and always sample 1000 vertices from each shape in the collection through all experiments. The intermediate refinement iterations are maintained only among the sampled points, and dense point-wise maps are only computed at the output stage.

In Section 6.1, we augment the dimension of functional maps from 30×30 to 80×80 with step size of 2, while in Section 6.2, we augment from 20×20 to 100×100 with step size of 5.

- Geodesic Distances for accuracy, bijectivity, runtime → also model evaluations?
- Can I use the dataset they have provided in the demo (because they also provide the initial maps for it) or do I need to run it on the FAUST, SHREC'07, etc?

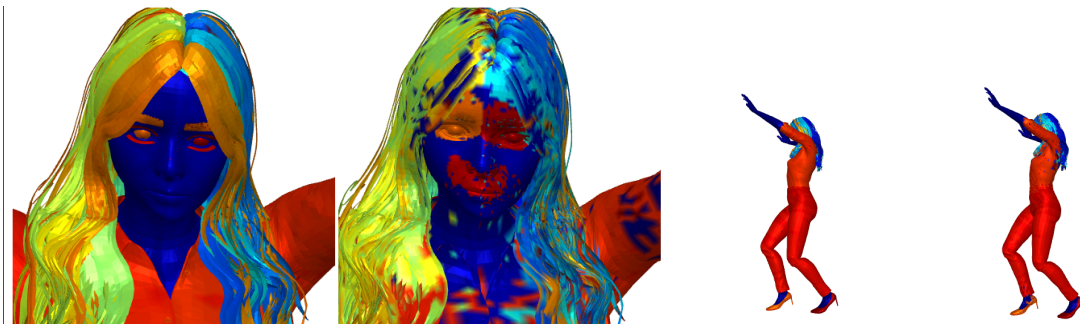
TO DO (11/09/2025):

- try pyFM <https://github.com/RobinMagnet/pyFM>
- run consistent zoomout
 - on their data
 - on our data (original mesh)
 - on the pointcloud (depth estimations)
 - Use the pointwise correspondence obtained through closest points for generating the initialisation

For Reference:

- Look at page 13 and 48
https://www.lix.polytechnique.fr/~maks/fmaps_SIG17_course/notes/siggraph17_course_notes.pdf
- Page 12 or 26
https://www.lix.polytechnique.fr/~maks/fmaps_SIG17_course/notes/siggraph17_course_notes.pdf
- Landmark point correspondences <https://people.csail.mit.edu/jsolomon/assets/fmaps.pdf>

Initial Pointwise map of frame 1 → frame 2 using KDTree nearest neighbour search from open3D



Ooops needed to use the vertices of mesh one rather than making a new colour map:



Next based on the ConsistentZoomout structure:

- I need to use pyFM to convert them to functional maps

- Embed them in a functional map network with all the maps and their basis functions right?
- Run with ConsistentZoomOut
- Then do the same for the depth point maps

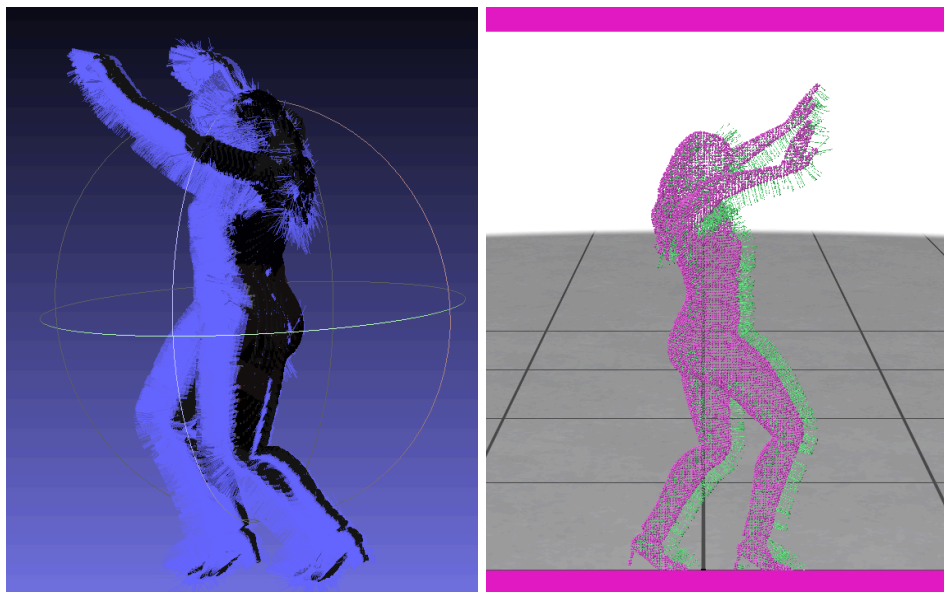
Another detail:

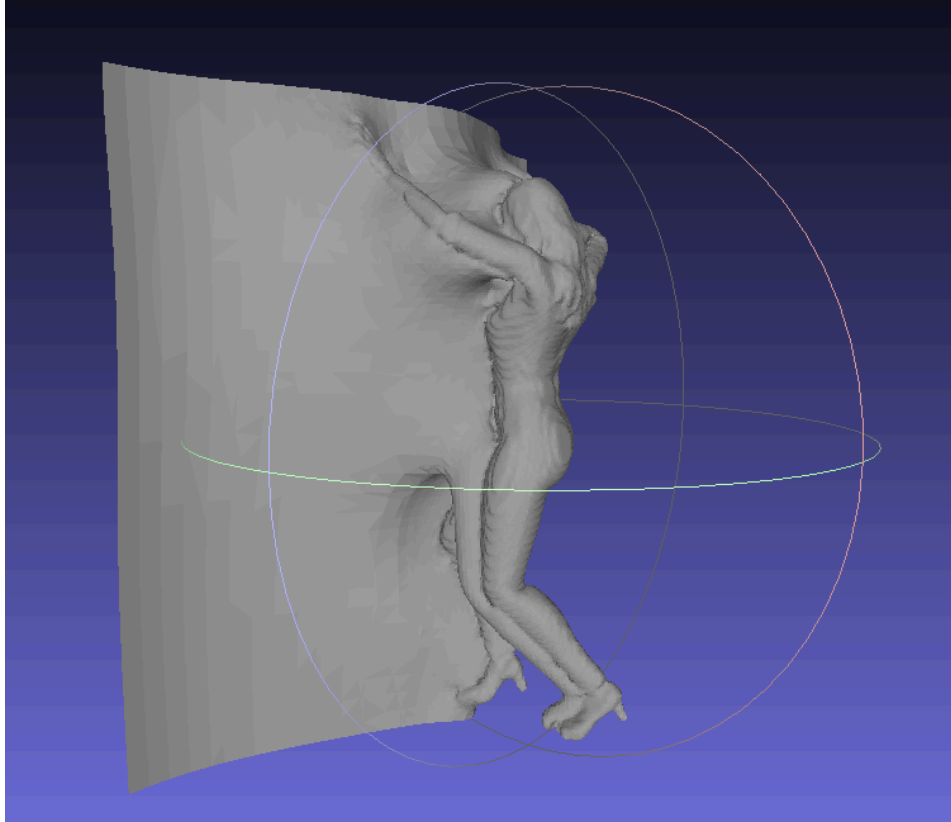
- The pointwise maps need to be from mesh 2 to mesh 1, and then the functional maps will be mesh 1 to mesh 2 right? So I need to reverse these ones?
- Also I'm still a bit confused about how pyFM is running ConsistentZoomOut

-
- Converting pointclouds to meshes:
https://www.open3d.org/docs/latest/tutorial/Advanced/surface_reconstruction.html
 - Remeshing to remove sections - check in blender?
 - Reconstruction with meshlab - Do I still need to convert to a trimesh form to use with consistent zoomout? - is there a way to do this more concisely?
 - Need to re-mesh the depth point clouds using normals:

Some Issues:

These are the pointclouds with normals. I'm sorry it took me so long to send, I've actually come across an issue with them where the normals seem to be pointing into the mesh from some of the camera angles and I can't figure out why. I did the normal estimation on the pointclouds from each of the camera separately before combining them into one cloud with open3d and the normals from all the views except the front view are pointing into the mesh, (the polyscope one below is from camera 2, points are pink and normals are green). I'm not sure if this is expected or if I am doing something wrong.

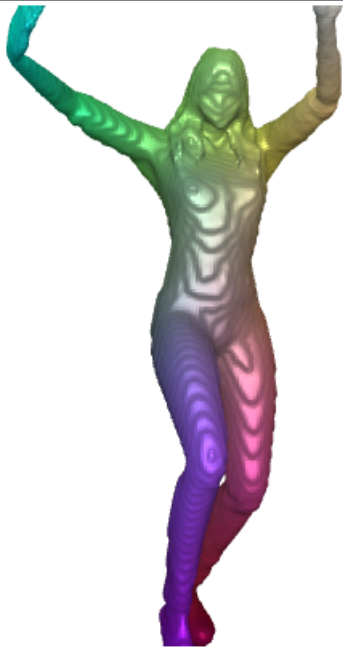




Progress So Far on Point Cloud version - as of 13/10/2025

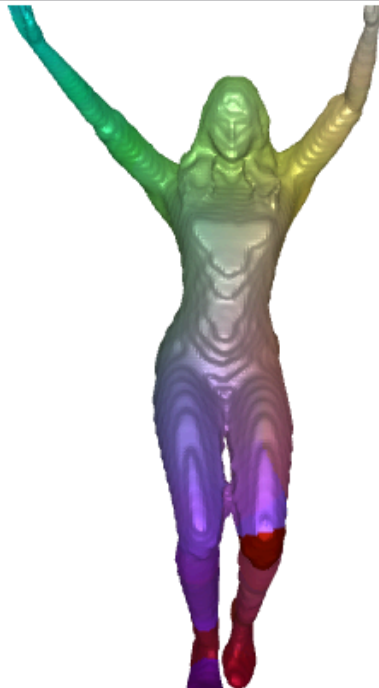
Using a FMN with 10 nodes and 90 edges if I remember correctly

Mesh 1 -> Mesh 10



After Running Consistent Zoomout

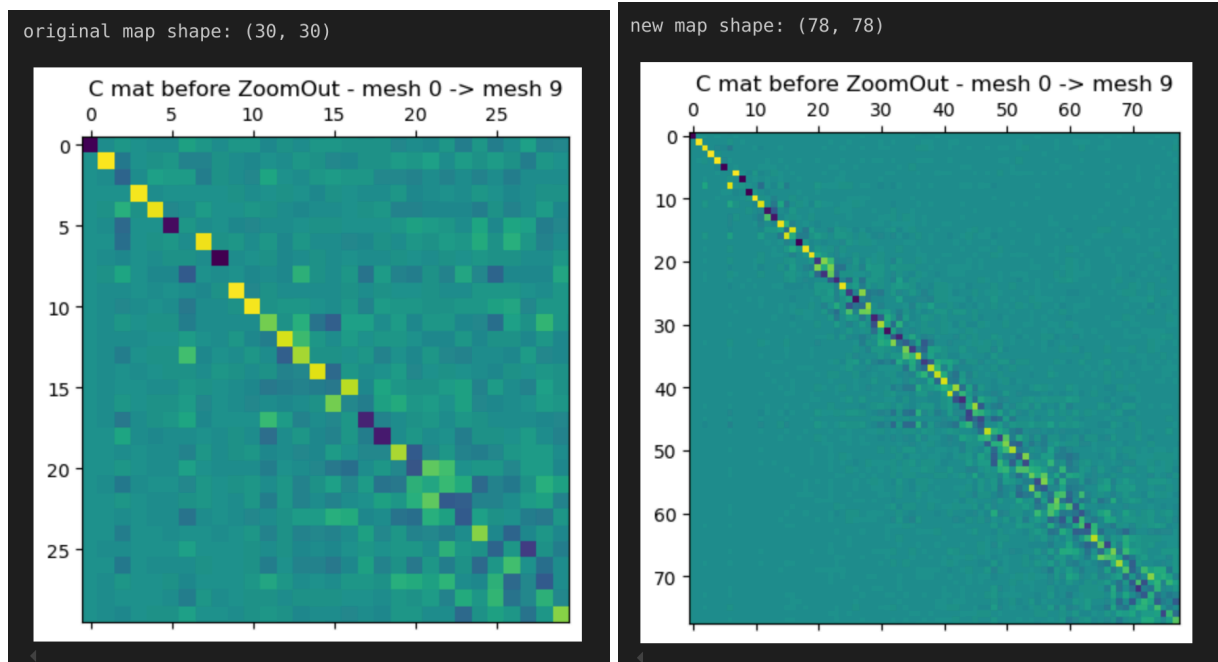
Mesh 1 -> Mesh 10



C Map Vis:

Using this: https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.matshow.html

Mesh 1 → Mesh 10:

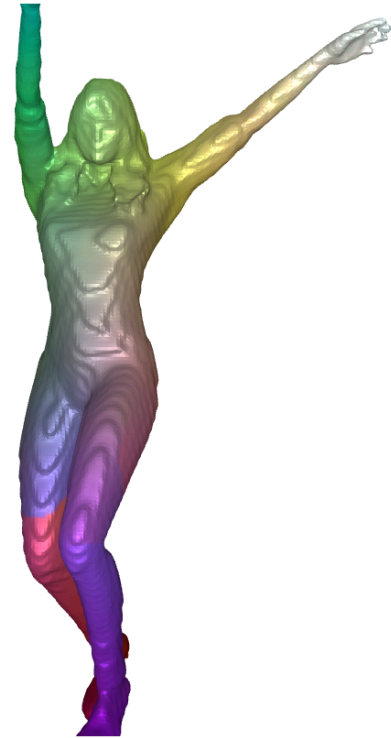
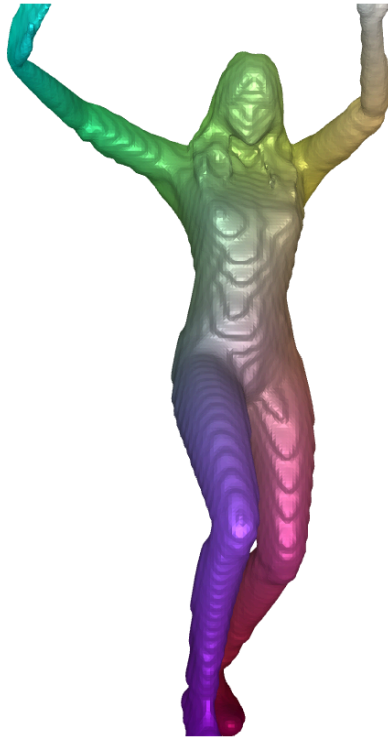


Using an FMN with 20 nodes and 380 edges.

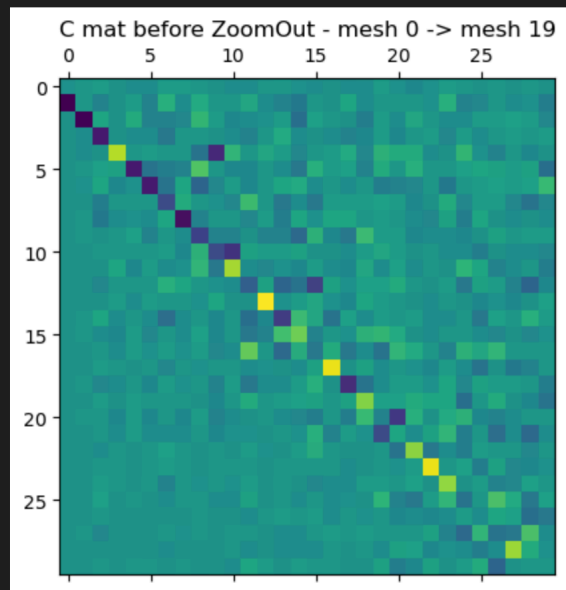
Mesh 1 → Mesh 20



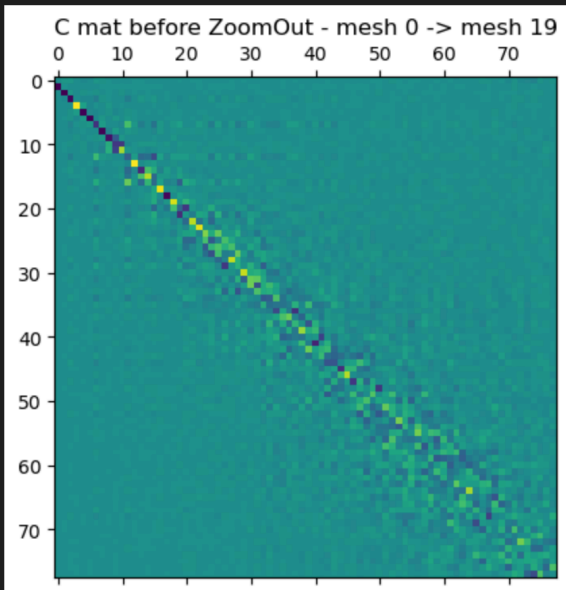
After CZ



original map shape: (30, 30)

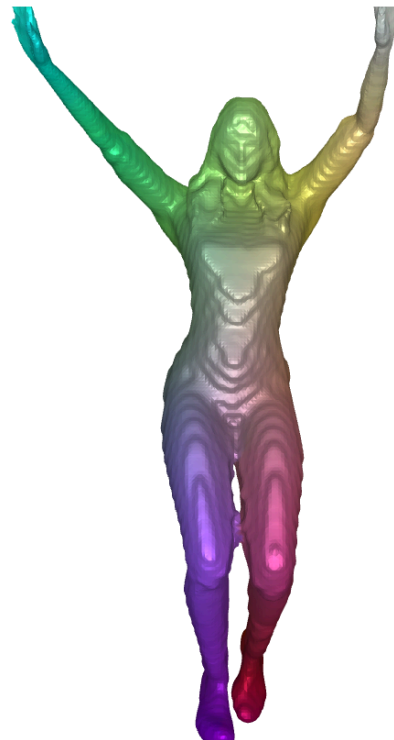
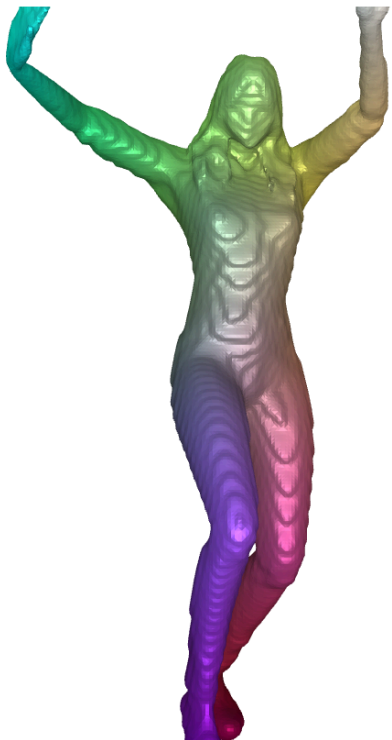


new map shape: (78, 78)

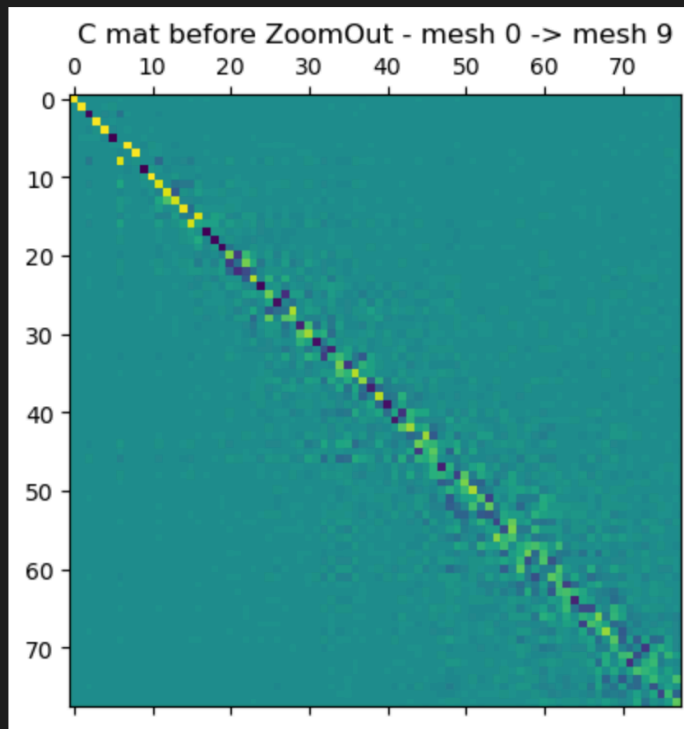


Using FMN with 20 nodes and 380 edges

New Mesh 1 → Mesh 10

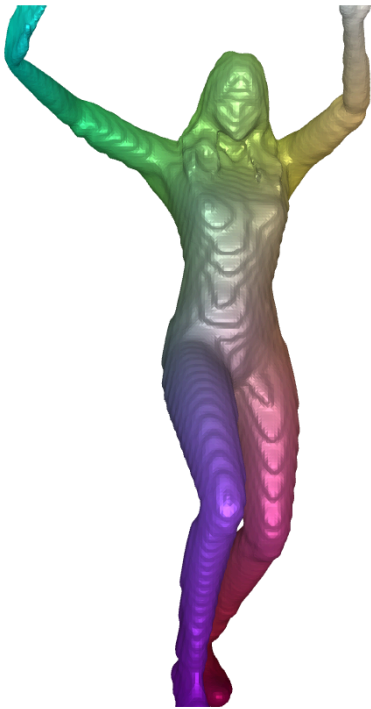


new map shape: (78, 78)



Some Issues:

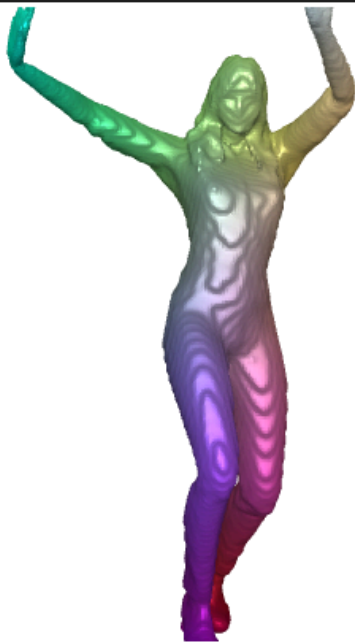
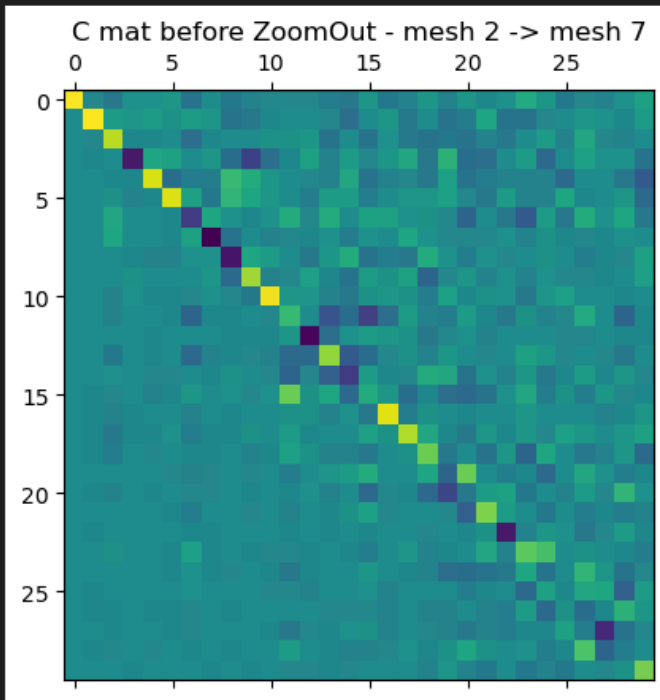
Visualisation Issues Still Happening!!!? Only sometimes



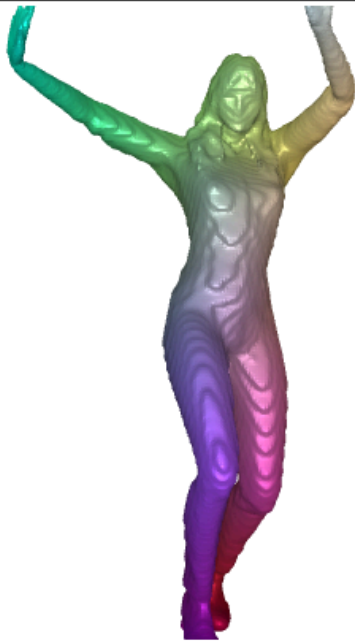
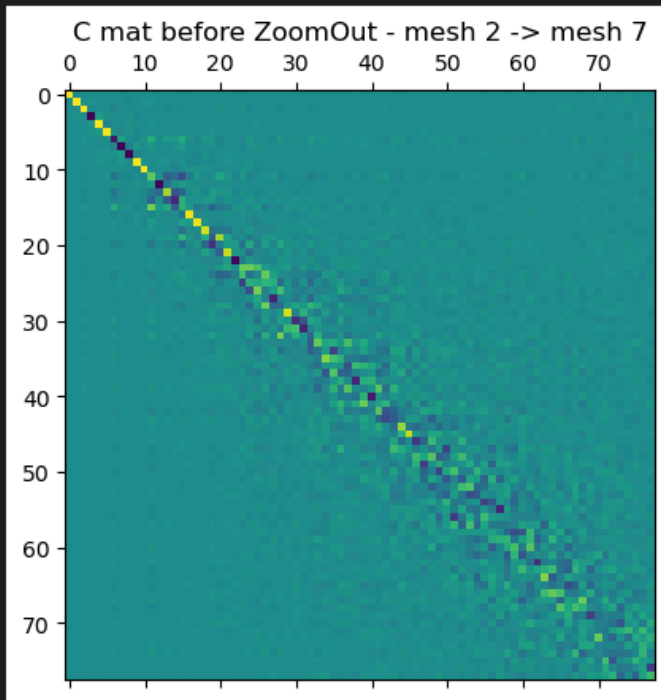
-
- For original consistent zoomout data - need to look at if pyFM is optimising the weights like they are doing in the demo

From the smaller dataset.

```
original map shape: (30, 30)
```



new map shape: (78, 78)

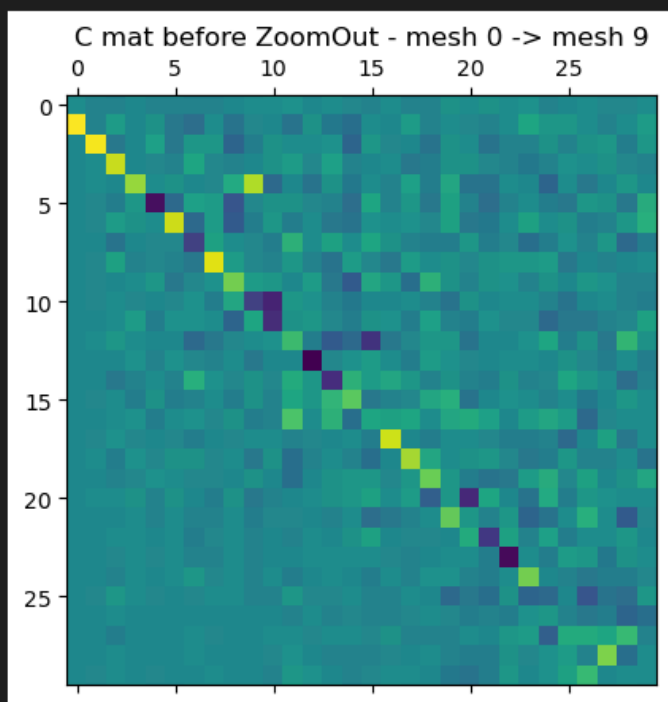


OK I feel like I've lost track of what am I actually trying to do

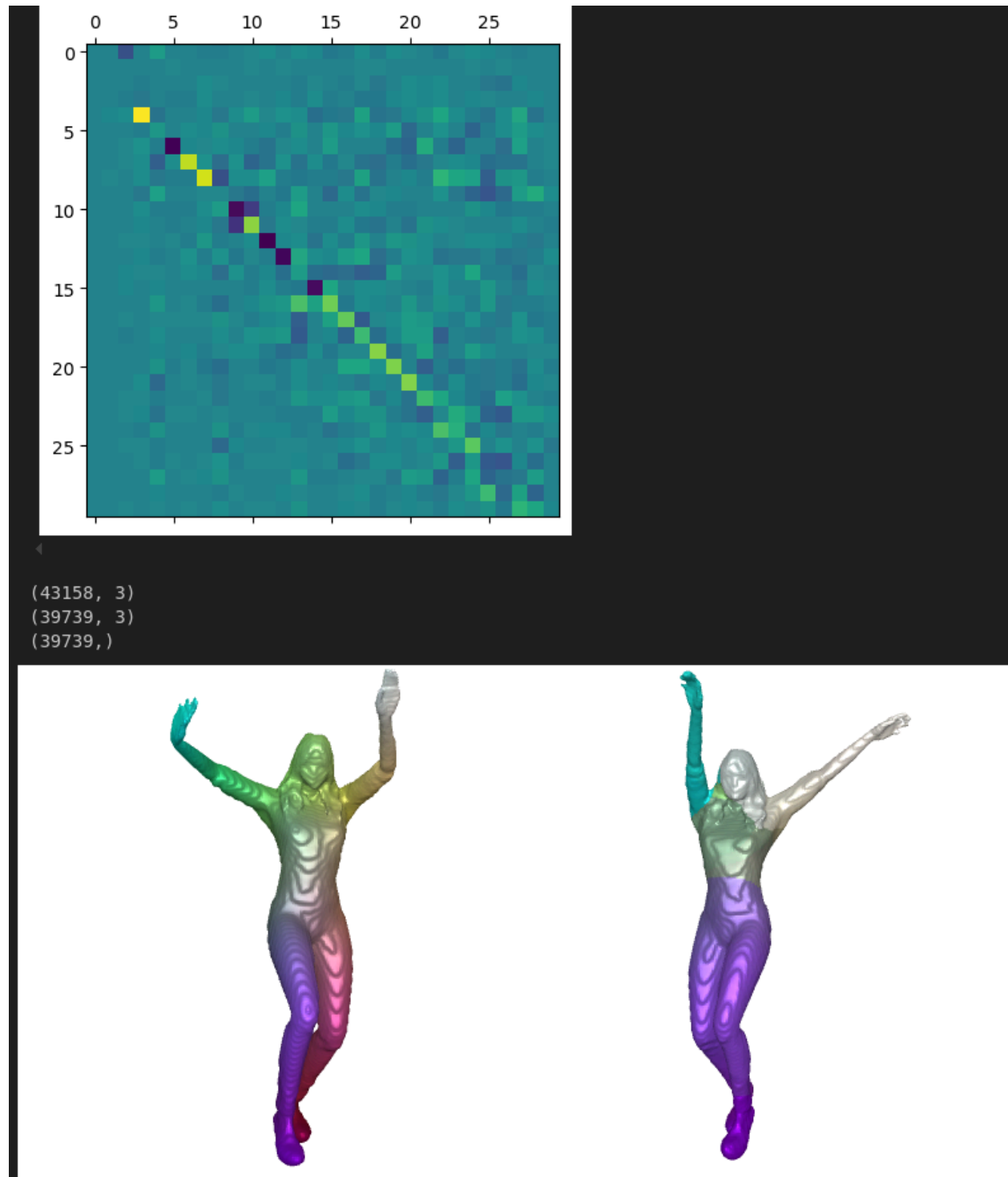
The point of using shape collections for shape correspondence is to use the pairwise maps between them to further refine all of the correspondences correct? Eventually I want to be able to apply this to video so If I have a shape collection of meshes from the frames of an animation (so sequential data), i.e. the collection consists of the same mesh in a different pose at each frame, I want to be able to refine the correspondences further than just using the current method of ConsistentZoomOut (2020) as implemented below by adding in a regularisation term based on time or sequence, sort of like if you know that frame 1 and 2 come just after each other then the correspondences between them would be better than the correspondences between 1 and 10. Basically I want to take the advantage of having a known linear sequence to the meshes to make the maps in the collection better. I am currently using pyFM.

how do I add this regularisation term without having to rework a lot of the work I've already done. Also my supervisor recommended using landmark based correspondence for creating the initial maps and the using the closest points for the regularisation somehow, what did he mean? and is it a viable method to achieve what I want?

Finally I am having trouble working out how to evaluate the method, how it is usually done? Do I need to generate ground truth maps? How? What's the quickest and most intuitive way to evaluate the results so that I know how well the method is working? How do these kinds of research usually do this?

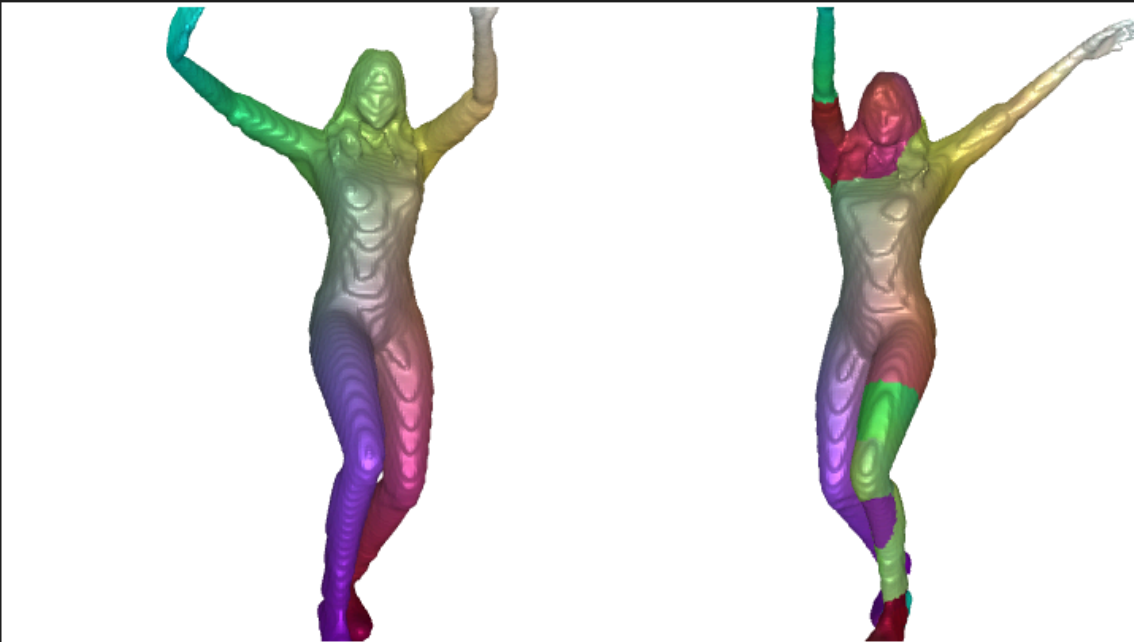
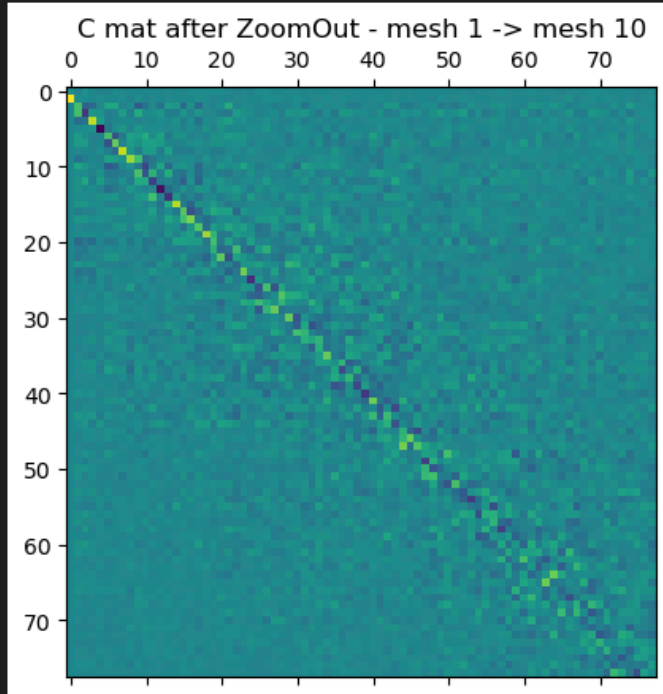


After Optimisation:

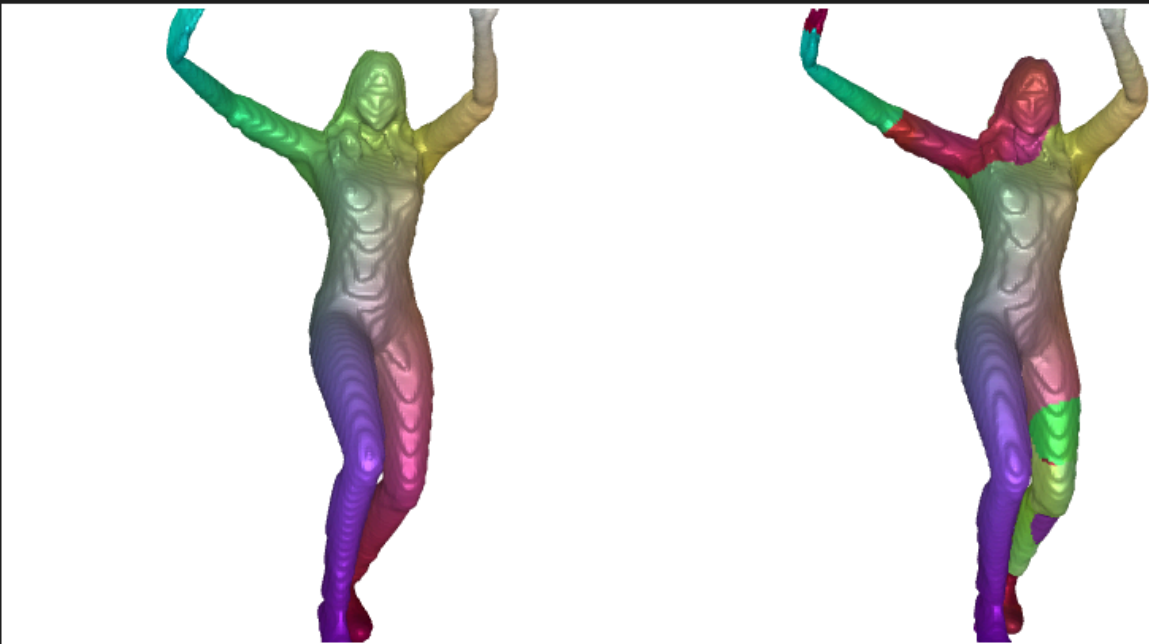
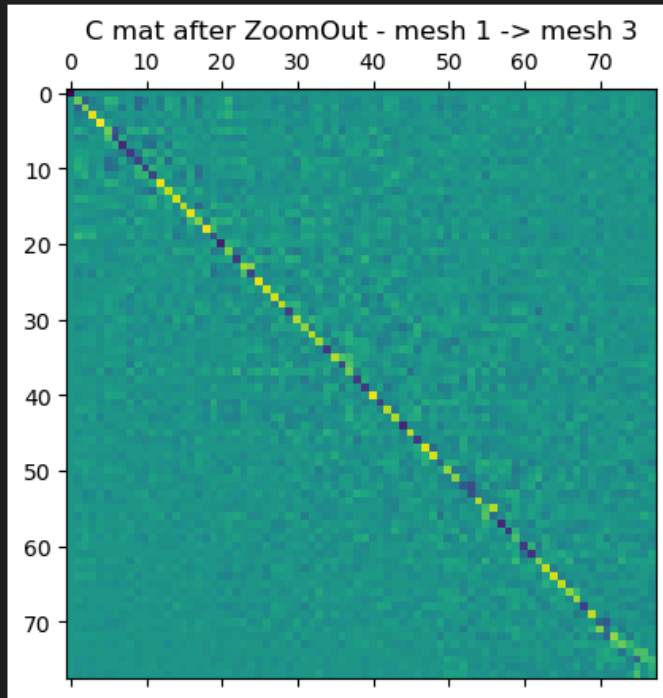


Its worse isn't it 😞

Yep its worse:



IT EVEN MADE THESE ONES WORSE!!!!

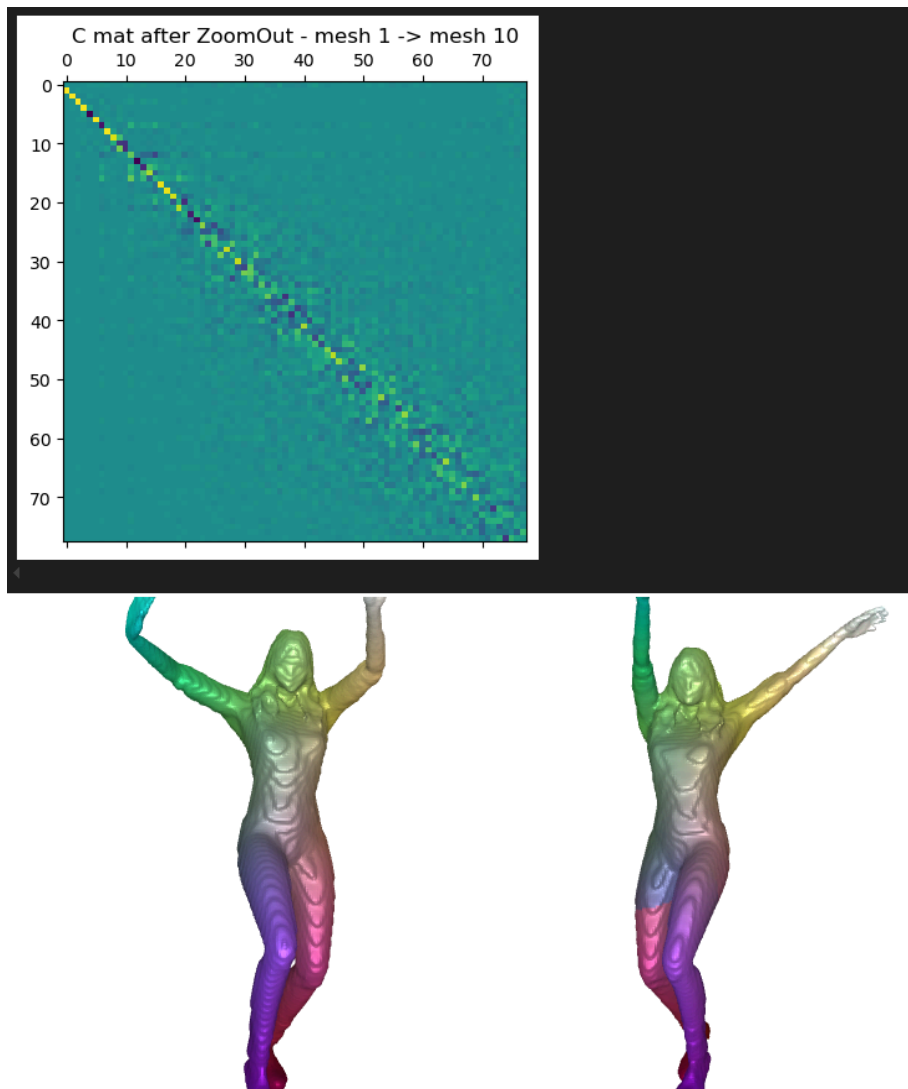


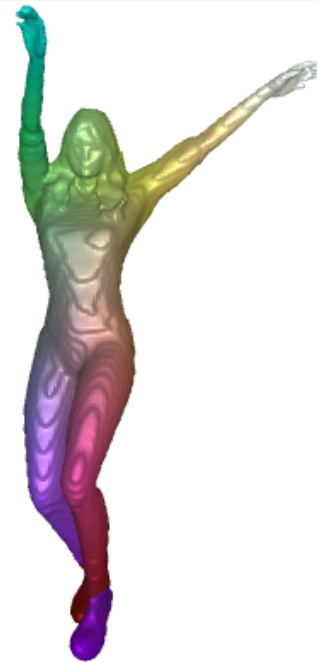
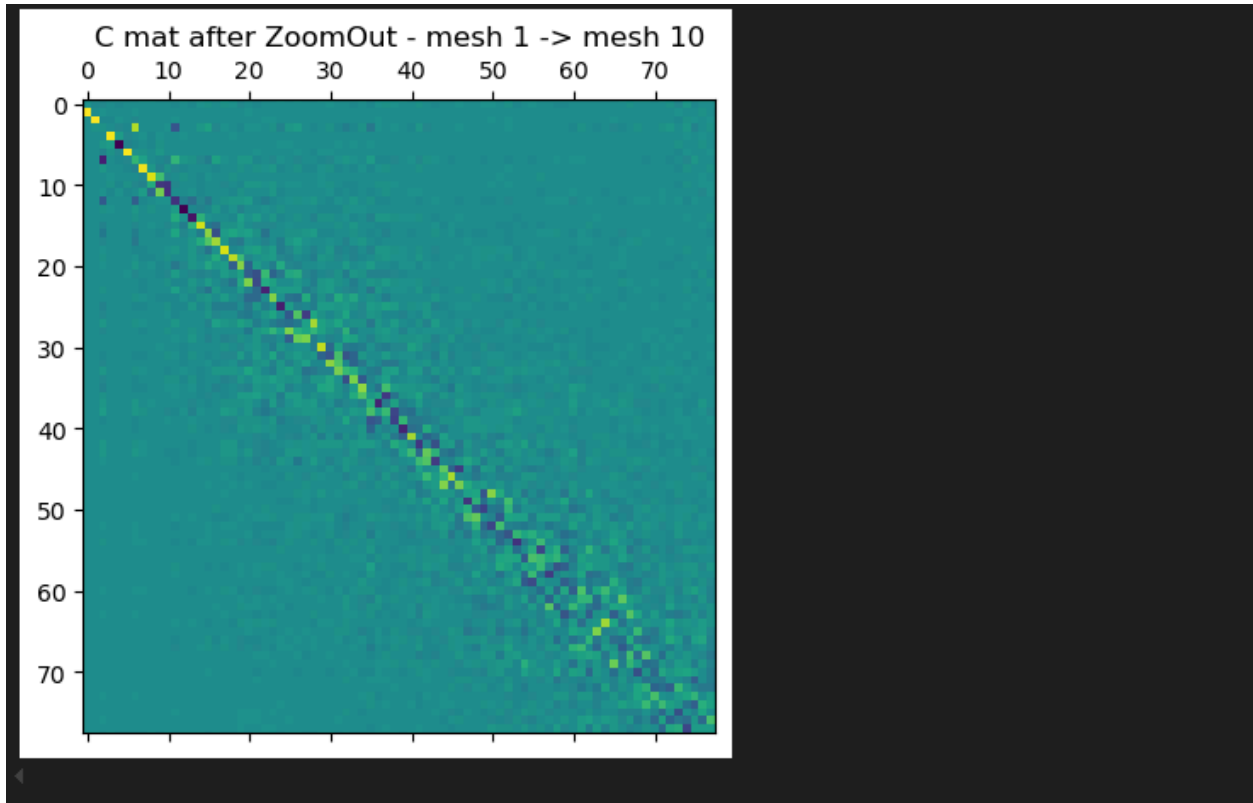
Possible message to ask Raphael:

Hi Raphael, hope you've had a good week so far. I need some help with the optimisation part, I think I might have misunderstood what you mentioned about using the intermediate frames.

So the way I'm doing it is if two frames are within a specific interval of time I get the initial maps like normal but if they are outside this interval I'll use an intermediate map to get the correspondence between them so instead of $T(i, j)$ its $T(i, x) @ T(x, j) = T(i, j)$. Then this will go through Zoomout. I tested it (the code is in the github in `consistent_zoomout/CZ_implementation_new_meshes.ipynb`, under the heading Optimisation), I'm not sure if its because I implemented it wrong or misunderstood something but its actually made the output worse 😞.

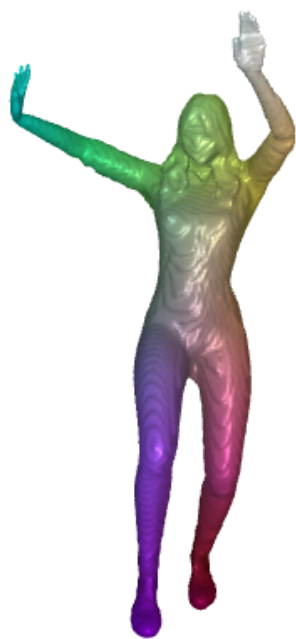
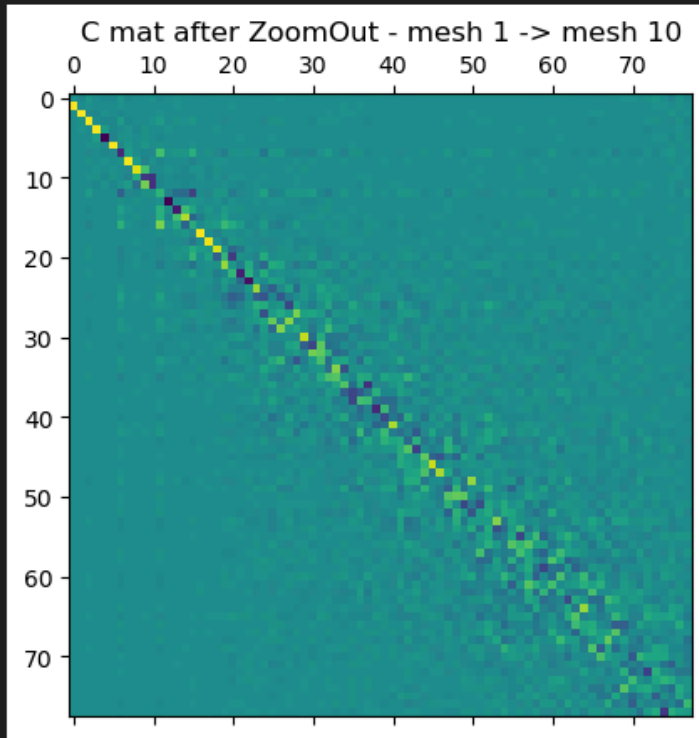
I just wanted to ask if I maybe misunderstood what you were saying about optimising based on time, is kind of manually doing it like this what you had in mind or did you mean more like including like a time-based regularisation term into the weights of the model.





^ is using adjacency so all edges weighted the same instead of icsm

new map shape: (78, 78)

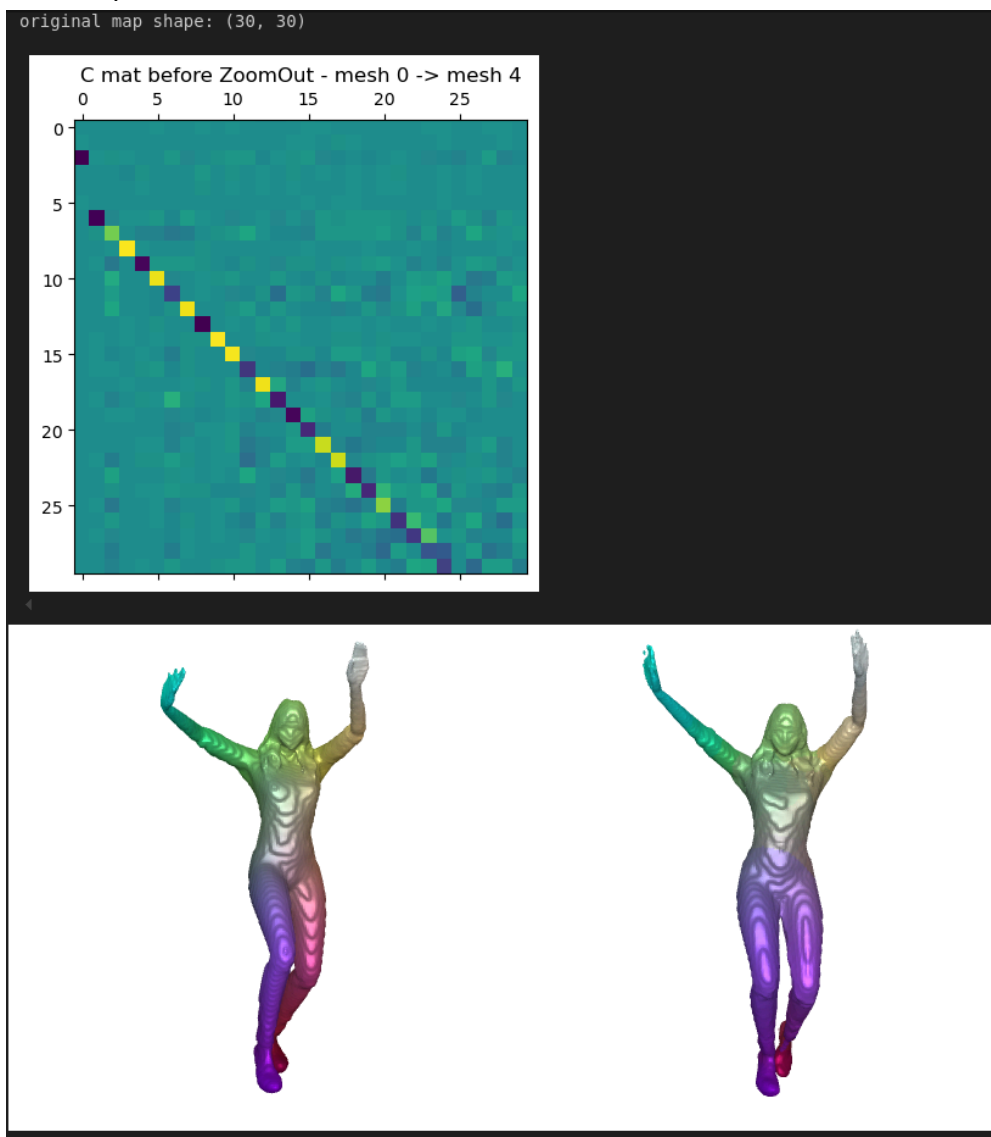


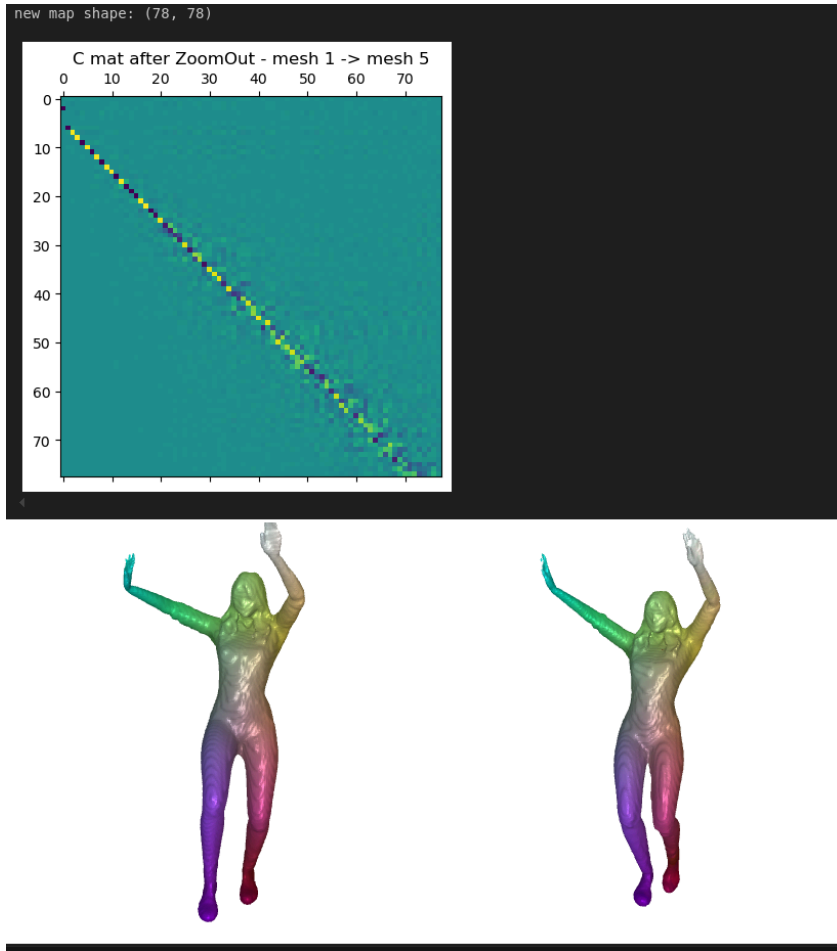
For decay:

$$w_{ij} = \begin{cases} 1, & |i - j| \leq \text{interval_size} \\ \exp(-\alpha(|i - j| - \text{interval_size})), & \text{otherwise} \end{cases}$$

- α controls how quickly weights drop as you move away from the interval.
 - Example: $\alpha = 0.5$ → moderate decay
 - $\alpha = 1.0$ → strong decay (closer edges dominate)
 - $\alpha = 0.2$ → smoother, long-range influence

Before optimisation





Inside FMN class set weights function:

```
#####
#                                     CHANGES ADDED HERE                                     #
#####
elif weight_type == 'sequence_decay':
    self.use_icsm = False # might incorporate icsm later once I read more about it in consistent zoomout paper

    if interval_size is None:
        interval_size = 2

    I = [x[0] for x in self.edges]
    J = [x[1] for x in self.edges]

    V = []
    for (i,j) in self.edges:
        val = abs(i - j)

        if val <= interval_size:
            weight = 1.0
        else:
            decay_factor = val - interval_size
            weight = 1.0/decay_factor

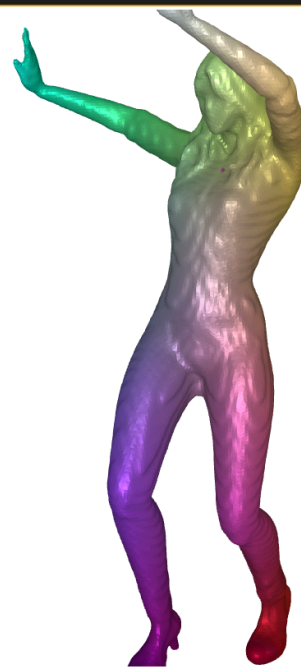
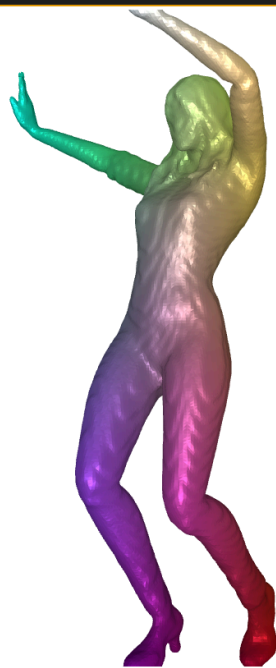
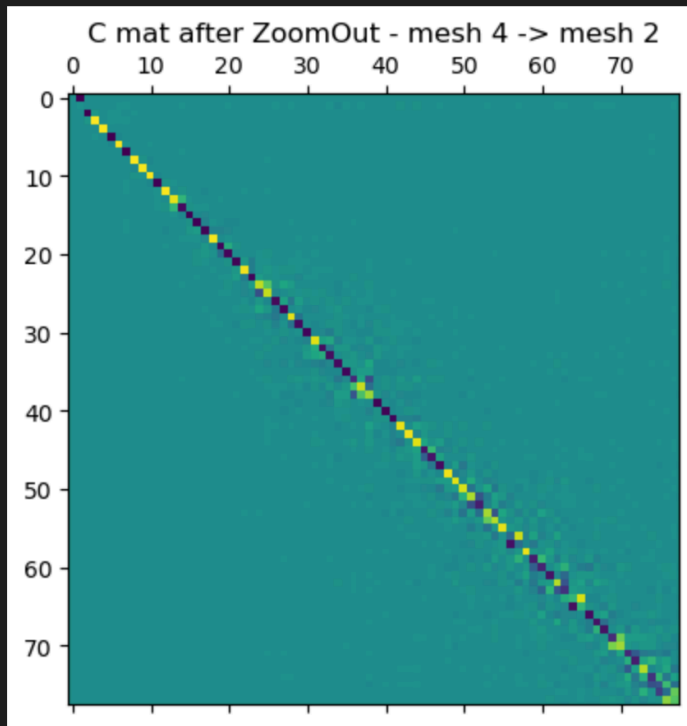
        V.append(weight)

    self.weights = sparse.csr_matrix((V, (I,J)), shape=(self.n_meshes, self.n_meshes))
    # undirected edges - look into this properly later I think the FM course talks about this in chapter 2
    self.weights = 0.5 * (self.weights + self.weights.T)

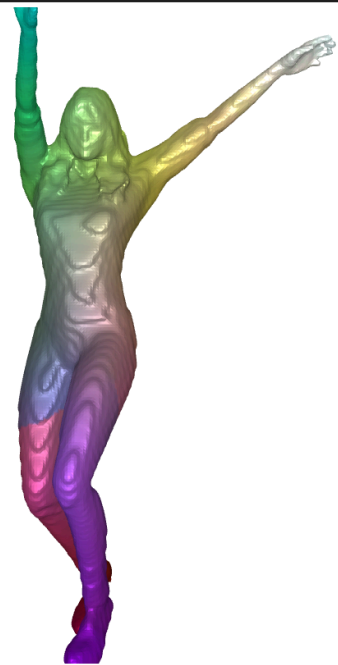
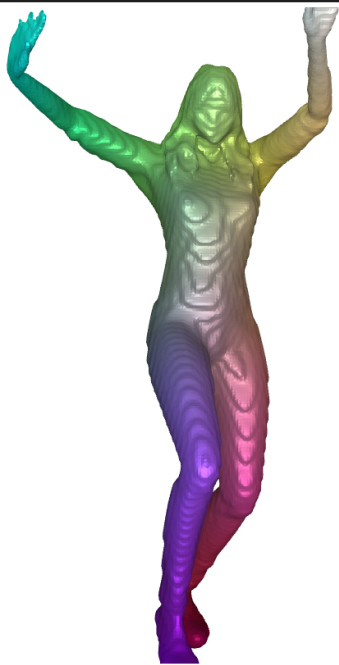
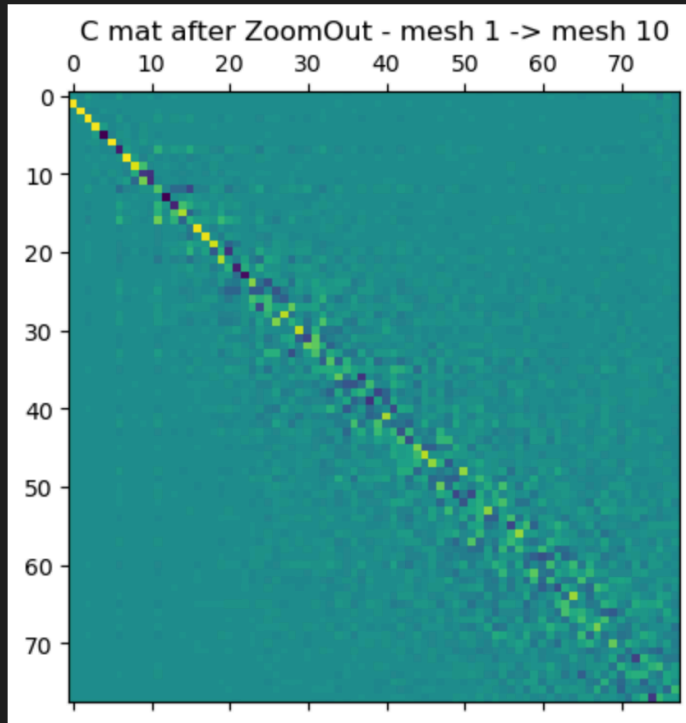
    # normalise
    self.weights /= np.max(V)
#####
```

With 10 meshes:

new map shape: (78, 78)



new map shape: (78, 78)



Current problem, the decay is not smooth, need to tune the paramedecay ters.

New Idea: Use icsm as the default and make sure it is preferred for meshes that are closer together in the sequence, then as they get further away in the sequence decay the impact of the weight but still apply icsm.

$$W_{ij} = \text{ICSM}_{ij} \times \underbrace{(1 + \beta e^{-\alpha |i-j|})}_{\text{temporal boost term}}$$

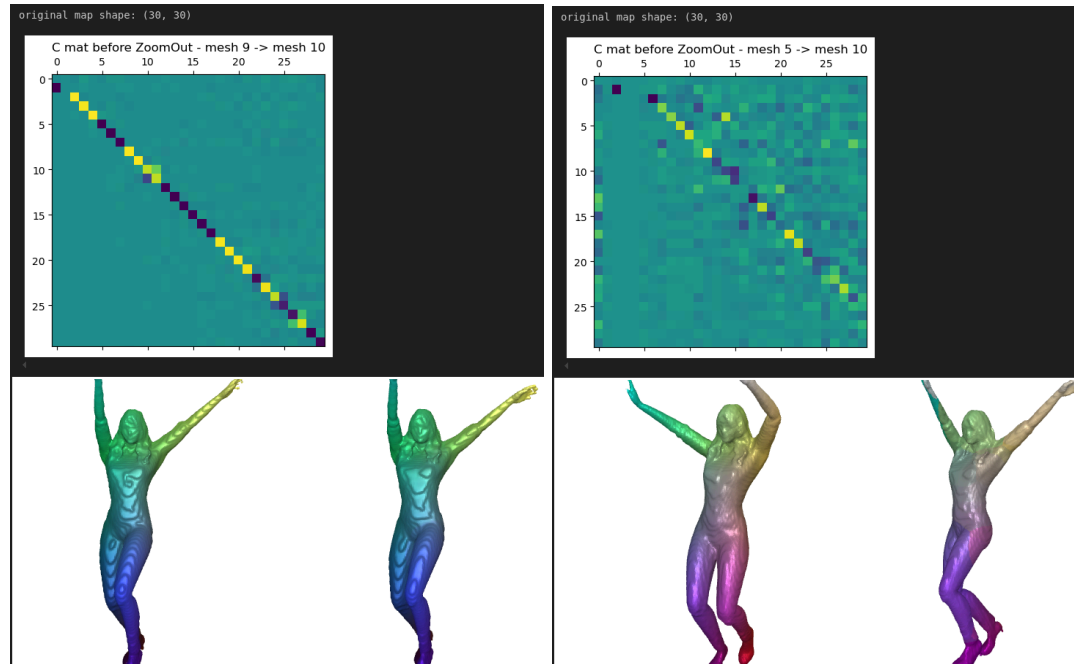
where:

- $\text{ICSM}_{ij} \rightarrow$ the normal cycle-consistency-based weight
- $\beta \rightarrow$ the maximum boost factor (e.g., 1.0 means up to $2 \times$ max weight)
- $\alpha \rightarrow$ how fast the boost decays with distance
- $|i - j| \rightarrow$ temporal distance between mesh indices

This way:

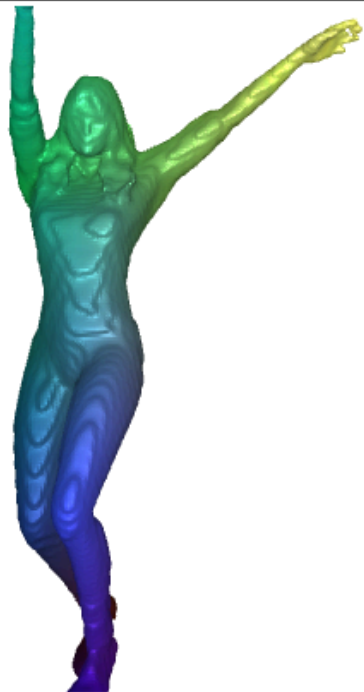
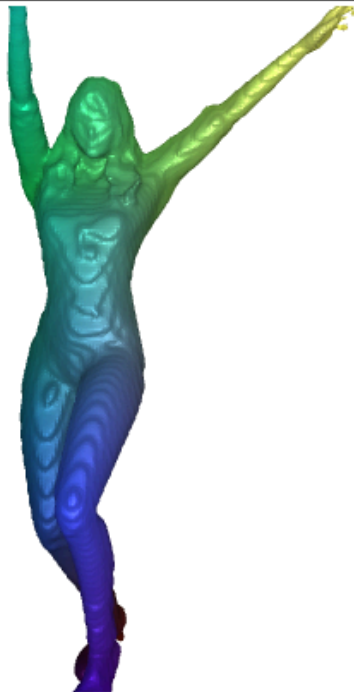
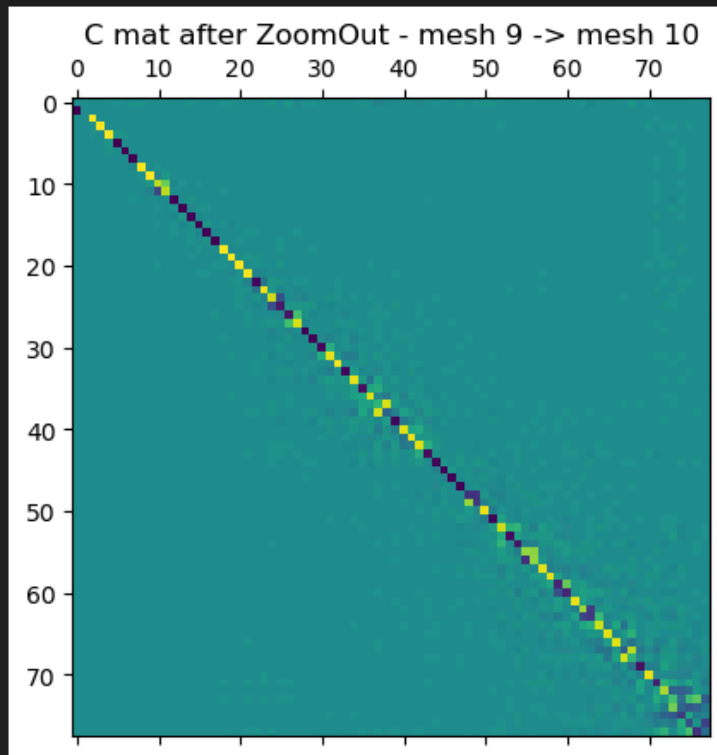
- Nearby meshes ($|i-j|$ small) $\rightarrow$ get up to $(1 + \beta) \times \text{ICSM}$
- Distant meshes $\rightarrow$ approach normal ICSM weights

Originals:

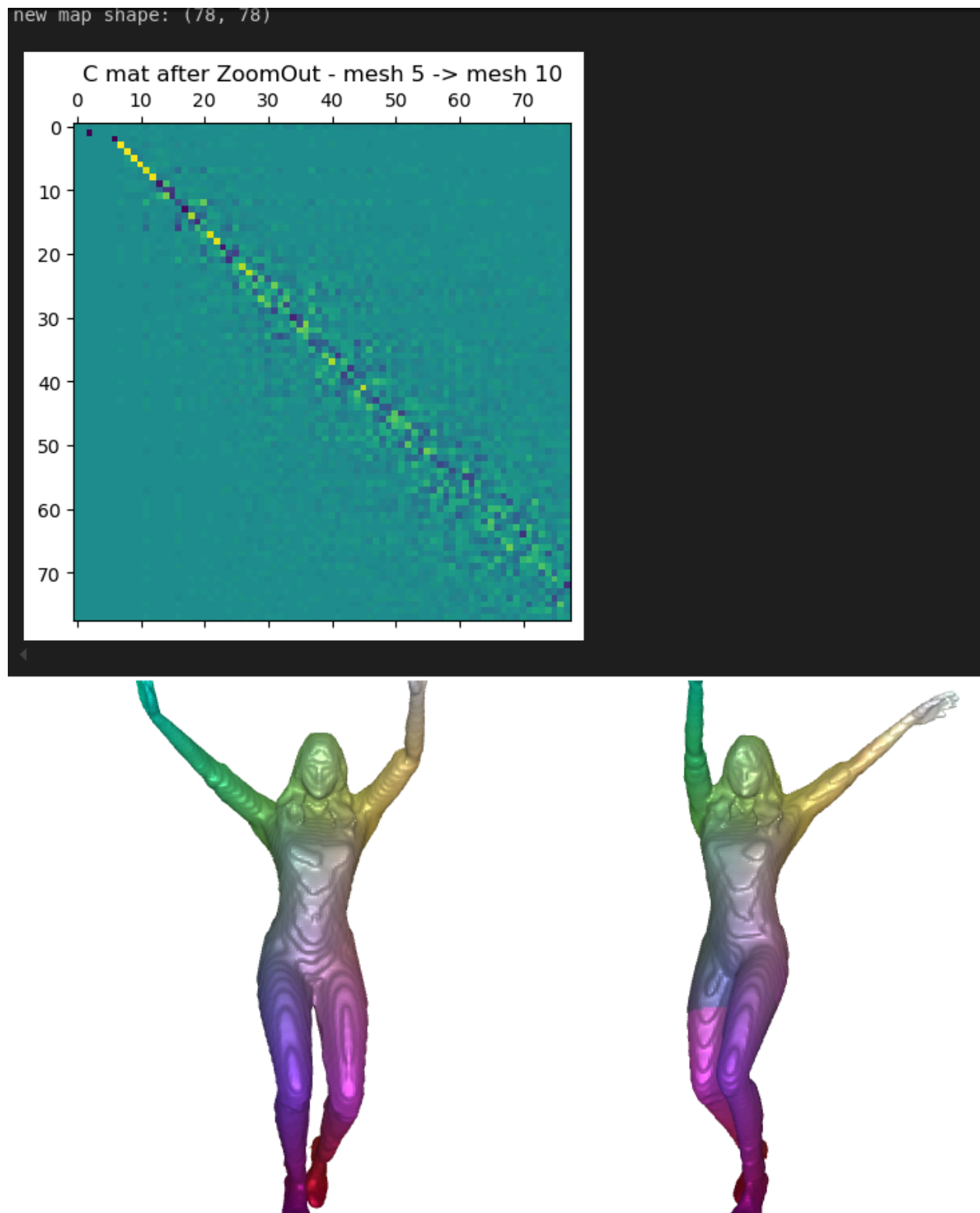


After Optimisation:

new map shape: (78, 78)



However:



Evaluation - as of 23/10/2025

To Do:

1. Implement Euclidean Distance from the pointcloud or mesh
2. Bijectivity constraints based on how CZ originally did it.

Important to know from the pyFM API Documentation.

`pyFM.spectral.convert.mesh_FM_to_p2p(FM_12, mesh1, mesh2, use_adj=False, subsample=None, n_jobs=1)`

Wrapper for FM_to_p2p using TriMesh class

Parameters:

- **FM_12** – (k2,k1) functional map in reduced basis
- **mesh1** ([TriMesh](#)) – source mesh for the functional map
- **mesh2** ([TriMesh](#)) – target mesh for the functional map
- **use_adj** (*bool*) – whether to use the adjoint map.
- **subsample** (*None or size 2 iterable ((n1'), (n2',))*) – Subsample of vertices for both mesh. If specified the p2p map is between the two subsamples.
- **n_jobs** (*int*) – number of parallel jobs. Use -1 to use all processes

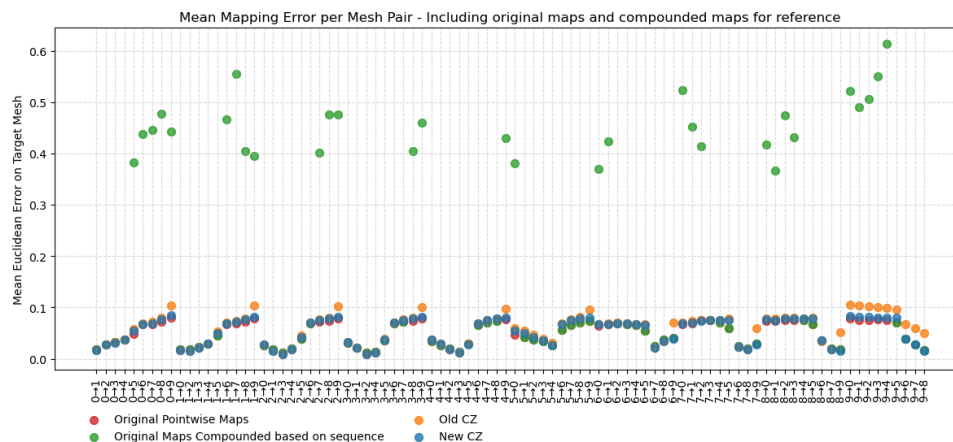
Returns:

p2p_21 – (n2,) match vertex i on shape 2 to vertex p2p_21[i] on shape 1

Return type:

`Np.ndarray`

Then there is this plot:



Processor	12th Gen Intel® Core™ i7-12800H × 20
Graphics	NVIDIA Corporation / Mesa Intel® Graphics (ADL GT2)

